



RAPPORT COMPLET AUDIT DE SÉCURITÉ

Aurora Society — Application Web

CONFIDENTIEL — USAGE INTERNE UNIQUEMENT

Date du rapport	31 mars 2026
Version application	1.0 — développement actif
Périmètre	Backend Node.js/Express + Frontend React/Vite + Nginx
Score de sécurité	6.5 / 10 (cible : 8 / 10 en production)
Critiques résolus	2 / 3 (HTTPS restant à activer sur le serveur)
Hautes priorités	5 / 6 corrigés

1. Résumé Exécutif

Aurora Society est une application de gestion de profils patrimoniaux et de mise en réseau pour membres privés. Elle traite des données **hautement sensibles** (patrimoine, documents d'identité, transactions financières), ce qui impose un niveau de sécurité élevé.

Ce rapport couvre l'intégralité des mesures de sécurité implémentées, les corrections appliquées sur les vulnérabilités critiques et hautes, ainsi que la couverture de tests automatisés.

Domaine	Score	État & Observations
Authentification JWT + Cookies	8 / 10	Tokens httpOnly, TTL réduits, 2FA, WebAuthn
Hachage mots de passe	9 / 10	bcrypt SALT_ROUNDS=12, comparaison en temps constant
Chiffrement des données sensibles	5 / 10	AES-256-GCM implémenté — clé à configurer en prod
Infrastructure réseau (HTTPS)	6 / 10	TLS documenté — Let's Encrypt à activer sur serveur
Gestion des secrets	7 / 10	.gitignore en place, JWT_SECRET 128 chars renouvelé
Rate limiting	7 / 10	Limiteurs dédiés /login, /register, /2fa, /password/forgot
Validation des entrées	7 / 10	express-validator, politique mot de passe renforcée
Protection CORS	8 / 10	Liste blanche testée, origines malveillantes bloquées
Numéros de compte	9 / 10	AU + 20 hex aléatoires (80 bits d'entropie)
Uploads fichiers	7 / 10	20 MB max, MIME+extension validés, octet-stream retiré
Tests de sécurité automatisés	6 / 10	9 fichiers de tests, couverture OWASP partielle

Score global : 6.5 / 10 — Objectif production : **8 / 10** — Les 3 actions bloquantes restantes : HTTPS, chiffrement champs sensibles, rotation secrets intégrations.

2. Mesures de Sécurité Implémentées

2.1 Authentification & Gestion de Session

Fichier : backend/src/middleware/auth.ts — backend/src/controllers/authController.ts

Mécanisme	Détail
Vérification JWT stricte	Types access / refresh / 2fa_pending — rejet avec code d'erreur distinct (TOKEN_EXPIRED, TOKEN_INVALID, TOKEN_MISSING)
Cookies httpOnly	aurora_access_token (15 min), aurora_refresh_token (7j), aurora_session (non-httpOnly, détection session), aurora_2fa_pending (10 min)
Attributs cookies	httpOnly=true, secure=true (prod), sameSite=lax (dev) / none (prod cross-subdomain)
Fallback Bearer token	Lecture cookie en priorité, header Authorization en secours (compatibilité clients API)
Vérification utilisateur	Existence + statut désactivé vérifiés en base après validation du JWT
Révocation sessions	Table RefreshToken avec jti unique, IP, User-Agent — révocation unitaire ou globale via /auth/logout-all

2.2 Authentification à Deux Facteurs (2FA)

- Code OTP 6 chiffres avec TTL 10 minutes
- Stocké en base PostgreSQL (table two_factor_codes) avec HMAC + expiration — plus de Map en mémoire
- Vérification du type de token 2fa_pending avant validation
- Cookie httpOnly aurora_2fa_pending — inaccessible au JavaScript
- Token d'accès complet émis uniquement après validation 2FA réussie

2.3 WebAuthn / Authentification Biométrique

- Clé publique stockée en base (table BiometricSession)
- Clé privée jamais transmise ni stockée côté serveur
- Horodatage lastUsedAt sur chaque session biométrique

2.4 Hachage des Mots de Passe

Fichier : backend/src/services/authService.ts

- bcryptjs avec SALT_ROUNDS = 12 (standard recommandé OWASP)
- Comparaison via bcrypt.compare() — résistant aux attaques par timing
- Jamais le mot de passe en clair en base de données
- [object Object][object Object]

2.5 Validation des Entrées

Fichier : backend/src/controllers/authController.ts + express-validator v7.2.0

- express-validator sur toutes les routes d'inscription/connexion
- Validation email avec normalisation (normalizeEmail())
- trim() sur tous les champs texte libres
- Rejet des requêtes invalides avant traitement

2.6 Protection des Routes Admin

Fichier : backend/src/routes/adminRoutes.ts

- Double protection : authMiddleware + requireRole('admin')
- Vérification du rôle en base de données (pas codé en dur)
- Toutes les routes admin protégées : stats, membres, logs, paramètres, backups

2.7 ORM Prisma — Protection Injection SQL

- Requêtes paramétrées par défaut — aucune concaténation de chaînes SQL
- UUID pour tous les identifiants (pas d'auto-incrément prédictible)
- Email unique en base (@unique) — protection contre la duplication
- Numéros de compte : format AU + 20 hexadécimaux aléatoires (80 bits d'entropie)

2.8 Headers de Sécurité HTTP

Fichier : backend/src/app.ts (helmet) + aurora-app/nginx.conf (Nginx)

Header	Valeur	Source
Content-Security-Policy	script-src 'self' + Stripe	helmet (API)
X-Frame-Options	SAMEORIGIN	Nginx (frontend)
X-Content-Type-Options	nosniff	Nginx + helmet
Referrer-Policy	strict-origin-when-cross-origin	Nginx
Permissions-Policy	geolocation=(), mic=(), camera=()	Nginx
X-XSS-Protection	1; mode=block	Nginx
Strict-Transport-Security	31536000; includeSubDomains	À activer via HTTPS

2.9 Rate Limiting

Fichier : backend/src/app.ts + backend/src/routes/authRoutes.ts

Route	Limite	Fenêtre	Comportement
Global API	2 000 req.	15 min	Production
/auth/login	10	15 min	Ignore succès
/auth/register	5	1 heure	Anti-spam inscriptions
/2fa/verify	5	10 min	Anti-brute OTP

/auth/password/forgot	5	15 min	Anti-énumération
-----------------------	---	--------	------------------

2.10 Sécurité des Uploads

- Taille max : 20 MB (réduit de 50 MB)
- application/octet-stream retiré des types MIME autorisés (empêchait le bypass)
- Validation croisée MIME type + extension de fichier
- Types autorisés : image/jpeg, image/png, image/webp, image/gif, application/pdf, audio/*

2.11 Gestion des Erreurs & Logs

- Rédaction automatique des champs password, token, jwt dans les logs
- Messages d'erreur génériques au client — stack trace jamais exposée
- auditService : login_failed, role_denied, admin_unauthorized/forbidden

2.12 Webhooks Signés (Veriff)

Fichier : backend/src/controllers/identityVerificationController.ts

- Signature HMAC vérifiée sur chaque webhook entrant
- Corps brut lu avant parsing JSON pour validation correcte de la signature
- Rejet avec 401 si signature invalide

2.13 Routes Frontend Protégées

Fichier : aurora-app/src/components/ProtectedRoute.tsx

- ProtectedRoute enveloppe toutes les routes membres et advisor
- AdminRoute pour les routes /admin/* (vérification rôle admin)
- Détection de session via cookie aurora_session (non-httpOnly, lisible JS)

3. Corrections des Vulnérabilités Critiques

Ces 3 vulnérabilités représentaient un **risque immédiat** pour la sécurité des données et des sessions utilisateurs.

Sévérité : CRITIQUE**CRIT-01 — Secrets exposés dans le dépôt git****CORRIGÉ**

Problème identifié

- JWT_SECRET avec valeur par défaut (your-super-secret-jwt-key-change-in-production)
- JWT_EXPIRES_IN = 7 jours (trop long, fenêtre d'exploitation étendue)
- Fichier .env non présent dans .gitignore du frontend
- Mots de passe Gmail et clés Stripe potentiellement commités

Corrections appliquées

- JWT_SECRET remplacé par 128 caractères hexadécimaux aléatoires (64 octets — crypto.randomBytes)
- JWT_EXPIRES_IN : 7 jours → 15 minutes
- JWT_REFRESH_EXPIRES_IN : 30 jours → 7 jours
- .env ajouté dans .gitignore (racine + aurora-app/)
- .env.example mis à jour avec instructions de génération des secrets

Actions complémentaires requises

- Vérifier l'historique git : git log --all -- "*.env"
- Réinitialiser le mot de passe Gmail / créer un app password dédié
- Réinitialiser les clés Stripe depuis le dashboard Stripe

Sévérité : CRITIQUE**CRIT-02 — Tokens JWT en localStorage (vulnérabilité XSS)****CORRIGÉ**

Problème identifié

Les tokens access et refresh étaient stockés dans **localStorage**, accessible par tout JavaScript de la page. Une attaque XSS (injection via bibliothèque tierce ou champ non échappé) permettait de voler les tokens et d'usurper l'identité de l'utilisateur.

Solution implémentée — Cookies httpOnly

Fichier	Avant	Après
authService.ts	<code>localStorage.setItem('token', ...)</code>	Lecture cookie <code>aurora_session</code> (non-httpOnly)
api.ts	<code>Authorization: Bearer \${token}</code>	<code>credentials: 'include'</code> (cookies auto)
authController.ts	Tokens dans le corps JSON	<code>res.cookie()</code> httpOnly Set-Cookie
auth.ts middleware	Lecture <code>Authorization</code> header uniquement	Cookie en priorité, Bearer en fallback

Utilitaires upload	<code>localStorage.getItem(ACCESS_TOKEN_KEY)</code>	<code>credentials: 'include'</code> uniquement
--------------------	---	---

Cookies configurés

Cookie	TTL	httpOnly	Secure	SameSite
aurora_access_token	15 min	Oui	Prod	lax / none (prod)
aurora_refresh_token	7 jours	Oui	Prod	lax / none (prod)
aurora_session	7 jours	Non	Prod	Lisible JS (détection)
aurora_2fa_pending	10 min	Oui	Prod	lax / none (prod)

Sévérité : CRITIQUE**CRIT-03 — Pas de HTTPS — HTTP uniquement****À FAIRE**

Problème

Les configurations Nginx actuelles n'écotent que sur le port 80 (HTTP). Toutes les communications transitent en clair : tokens, mots de passe, données patrimoniales, documents.

Configuration cible (Let's Encrypt + Certbot)

```
# Installation Certbot
sudo apt install certbot python3-certbot-nginx

# Obtenir les certificats (backend + frontend)
sudo certbot --nginx -d aurorasociety.ch -d back.aurorasociety.ch

# Certbot configure automatiquement :
#   - Bloc server 443 avec SSL/TLS
#   - Redirection HTTP → HTTPS (301)
#   - Renouvellement automatique (cron)
```

Headers à ajouter une fois HTTPS actif

- Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
- Activer les cookies Secure (déjà conditionnel à IS_PROD dans authController.ts)
- Activer SameSite=none pour les cookies cross-subdomain (app.aurorasociety.ch ↔ back.aurorasociety.ch)

4. Corrections de Priorité Haute

Sévérité : HAUTE**HIGH-01 — Durée de vie JWT trop longue****CORRIGÉ**

- Access token : 7 jours → 15 minutes (-99%)
- Refresh token : 30 jours → 7 jours (-77%)
- Mécanisme de refresh automatique côté frontend déjà en place (authService.ts)

Sévérité : HAUTE**HIGH-02 — Rate limiting insuffisant sur routes sensibles****CORRIGÉ**

- Limiteurs dédiés implémentés dans authRoutes.ts (voir tableau section 2.9)
- skipSuccessfulRequests: true sur /auth/login (ne compte pas les connexions réussies)
- Standardisation des messages d'erreur 429

Sévérité : HAUTE**HIGH-03 — 2FA stocké en mémoire (non distribué)****CORRIGÉ**

- Migration de Map en mémoire → table PostgreSQL two_factor_codes
- HMAC sur les codes stockés + champ expiration
- Usage unique garanti (suppression après validation)
- Recommandation : Redis pour déploiement multi-instances

Sévérité : HAUTE**HIGH-04 — Données sensibles non chiffrées en base****EN COURS**

- Chiffrement AES-256-GCM implémenté dans le code (IV aléatoire + auth tag)
- Activé si la variable FIELD_ENCRYPTION_KEY est définie
- [object Object][object Object]

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

- Champs concernés : ProfilePrivate.phone, ProfilePrivate.patrimoine, données identité Veriff

Sévérité : HAUTE**HIGH-05 — Limites d'upload trop permissives****CORRIGÉ**

- Taille max : 50 MB → 20 MB
- application/octet-stream retiré (permettait le bypass du filtre MIME)
- Validation croisée MIME type + extension de fichier

Sévérité : HAUTE**HIGH-06 — Politique de mot de passe insuffisante****CORRIGÉ**

- Longueur minimum : 8 → 12 caractères
- Exigences ajoutées : majuscule, minuscule, chiffre, caractère spécial
- Implémenté dans passwordPolicy.ts — appliqué sur register et password change

5. Tests de Sécurité Automatisés

9 fichiers de tests sont présents dans **backend/src/__tests__**/. Les tests de sécurité couvrent les vecteurs d'attaque les plus courants de l'OWASP Top 10.

5.1 Tableau de Couverture

Suite	Test	Résultat
securityHeaders	GET /health → Content-Security-Policy présent	PASS
securityHeaders	GET /auth/me sans token → 401 + code TOKEN_MISSING	PASS
security.integration	Brute force /auth/login → 429 après 10 tentatives	SKIP (DB)
security.integration	Route /admin/stats → 403 pour utilisateur standard	SKIP (DB)
security.integration	Profil privé → 403 depuis un autre utilisateur (isolation données)	SKIP (DB)
security.integration	Injection SQL via findUnique — table User intacte	SKIP (DB)
security.integration	GET /profiles/:id avec payload SQL — pas de 200 ni crash	SKIP (DB)
corsOrigin	Production sans CORS_ORIGIN — origine malveillante non incluse	PASS
corsOrigin	Production avec CORS_ORIGIN restreint — uniquement origines listées	PASS
corsOrigin	Développement — origine publique malveillante → erreur CORS	PASS
corsOrigin	Développement — localhost:5173 autorisé	PASS
accountNumber	Format AU + 20 hex (80 bits d'entropie)	PASS
accountNumber	50 tirages successifs distincts (pas de collision)	PASS

5.2 Détail des Tests Clés

Brute Force Protection (security.integration.test.ts)

Scenarior : 10 tentatives de connexion échouées avec la même IP (header X-Forwarded-For simulé) → la 11ème doit retourner 429 (Too Many Requests).

```
it('bloque avec 429 après 10 tentatives', async () => {
  for (let i = 0; i < 10; i++) {
    await request(app)
      .post(`${prefix}/auth/login`)
      .set('X-Forwarded-For', fwd)
      .send({ email, password: 'WrongPassword1!Aa' });
  }
  const blocked = await request(app)
    .post(`${prefix}/auth/login`)
    .set('X-Forwarded-For', fwd)
    .send({ email, password: 'WrongPassword1!Aa' });
  expect(blocked.status).toBe(429);
});
```

Isolation des Données (security.integration.test.ts)

Scénario : l'utilisateur A (token valide) tente d'accéder au profil privé de l'utilisateur B → 403 Forbidden.

```
it("refuse l'accès au profil privé d'un autre utilisateur (403)", async () => {
  const token = accessToken(idA, emailA);
  const res = await request(app)
    .get(`${prefix}/profiles/user/${idB}`)
    .set('Authorization', `Bearer ${token}`);
  expect(res.status).toBe(403);
  expect(res.body?.error).toMatch(/non accessible/i);
});
```

Anti-Injection SQL (security.integration.test.ts)

Scénario : une chaîne malveillante (; DROP TABLE "User"; --) est passée à Prisma.findUnique. Le nombre de lignes en base doit rester identique.

```
it('findUnique avec chaîne malveillante ne vide pas la table User', async () => {
  const malicious = `'; DROP TABLE "User"; --`;
  const before = await prisma.user.count();
  try {
    await prisma.user.findUnique({ where: { id: malicious } });
  } catch { /* Prisma rejette le format invalide - acceptable */ }
  expect(await prisma.user.count()).toBe(before);
});
```

CORS (corsOrigin.test.ts)

Scénario : en production, l'origine https://evil.example ne doit jamais être dans la liste autorisée. En développement, la callback doit rejeter les origines publiques non listées.

Numéros de Compte Non-Prédictibles (accountNumber.test.ts)

Scénario : 50 tirages de numéros de compte au format AU + 20 hex doivent tous être distincts, prouvant l'absence de collision et la non-prédictibilité.

5.3 Tests SKIP (DB) — Conditions d'Exécution

Les tests marqués **SKIP (DB)** nécessitent une base PostgreSQL accessible (variable **DATABASE_URL** définie). Ils s'exécutent automatiquement en CI/CD avec la base de staging.

```
# Exécuter tous les tests (avec base de données)
DATABASE_URL="postgresql://user:pass@localhost:5432/aurora_test" npx jest

# Tests unitaires seuls (sans base)
npx jest --testPathIgnorePatterns="integration"
```

5.4 Tests à Ajouter (Recommandations)

Test	Objectif	Priorité
Rate limit 2FA (/2fa/verify)	Bloquer après 5 tentatives	Haute
Token expiré — refresh automatique	401 → refresh → retry réussit	Haute

Upload type MIME non autorisé	application/octet-stream → rejeté	Haute
CSP frontend (Nginx)	Header Content-Security-Policy présent	Moyenne
Énumération membres	IDs non séquentiels (UUID)	Moyenne
Accès admin sans rôle admin	403 Droits insuffisants	Haute

6. Plan d'Action

Phase 1 — Avant mise en production (IMMÉDIAT)

#	Action	Responsable / Fichier	Effort
1	Activer HTTPS avec Let's Encrypt sur le serveur VPS	Ops / nginx-*.conf	2 h
2	Configurer FIELD_ENCRYPTION_KEY (32 octets) et activer chiffrement AES	Backend .env + service	1 h
3	Réinitialiser Gmail app password + clés Stripe	Consoles Gmail / Stripe	30 min
4	Configurer NODE_ENV=production + VERIFF_WEBHOOK_SKIP_VERIFY=false	.env production	15 min

Phase 2 — Mois 1

#	Action	Responsable / Fichier	Effort
5	Activer CSP dans Nginx frontend (SPA React)	aurora-app/nginx.conf	2 h
6	Ajouter tests rate limit 2FA, upload, token refresh	__tests__ /	1 jour
7	Configurer backup automatique PostgreSQL	Ops / cron	3 h
8	Audit npm (npm audit fix) sans vulnérabilités critiques	package.json	1 h

Phase 3 — Mois 2-3

#	Action	Responsable / Fichier	Effort
9	Migrer rate limit vers Redis (distribué, multi-instances)	app.ts + rate-limit-redis	1 h
10	Enrichir logs d'audit (connexions réussies, changements MP, accès données)	auditLog.ts	1 jour
11	Tests OWASP complets (couverture >80% des vecteurs)	__tests__ /	3 jours
12	Restreindre accès DB (PostgreSQL non exposé publiquement)	Ops / firewall	2 h

7. Checklist de Mise en Production

À valider avant chaque déploiement en production :

Secrets & Configuration

- ✓ .env présent dans .gitignore (racine + aurora-app/)
- ✓ JWT_SECRET fort — 128 caractères hexadécimaux aléatoires (64 octets)
- ✓ JWT_EXPIRES_IN = 15m, JWT_REFRESH_EXPIRES_IN = 7d
- ❑ Clés Stripe réinitialisées (dashboard Stripe) — test en staging, live en prod
- ❑ SMTP_PASS réinitialisé — app password Gmail dédié, pas le mot de passe compte
- ❑ VERIFF_WEBHOOK_SKIP_VERIFY=false en production
- ❑ NODE_ENV=production en production
- ❑ FIELD_ENCRYPTION_KEY défini — 32 octets hex (chiffrement AES-256-GCM actif)

Réseau & TLS

- ❑ HTTPS activé (port 443) avec certificat Let's Encrypt valide
- ❑ Redirection HTTP → HTTPS configurée (301 permanent)
- ❑ HSTS activé : max-age=31536000; includeSubDomains
- ✓ CORS restreint aux domaines de production uniquement
- ❑ Cookies Secure=true actif (conditionnel IS_PROD — se déclenche avec NODE_ENV=production)

Authentification

- ✓ Tokens JWT dans des cookies httpOnly (plus de localStorage)
- ✓ Rate limiting sur /auth/login ≤ 10 tentatives / 15 min
- ✓ Rate limiting sur /auth/2fa/verify ≤ 5 tentatives
- ✓ 2FA codes en base PostgreSQL (plus de Map en mémoire)
- ✓ Politique mot de passe : 12 chars, majuscule, chiffre, spécial
- ✓ Numéros de compte non-séquentiels : AU + 20 hex aléatoires

Base de Données

- ❑ Données sensibles chiffrées (patrimoine, phone) — AES-256-GCM actif
- ❑ Backup automatique configuré et testé
- ❑ Accès PostgreSQL restreint au backend uniquement (non exposé publiquement)
- ❑ Migrations Prisma appliquées

Uploads

- ✓ Limite fichier ≤ 20 MB
- ✓ application/octet-stream retiré des types autorisés
- ✓ Validation croisée MIME type + extension de fichier
- ❑ Dossier uploads/ non accessible directement (CDN sécurisé ou accès contrôlé)

Frontend

- ✓ Aucun secret dans les variables VITE_*
- ✓ ProtectedRoute sur toutes les routes authentifiées (App.tsx)
- ✓ AdminRoute sur les routes /admin/*
- ❑ CSP dans Nginx frontend (Content-Security-Policy header)
- ❑ npm audit sans vulnérabilités critiques

Tests

- ✓ Tests headers sécurité passent (CSP, 401 sans token)
- ✓ Tests CORS passent (origines malveillantes bloquées)
- ✓ Tests numéros de compte passent (non-séquentiels)
- ❑ Tests brute force passent (429 après 10 tentatives)
- ❑ Tests autorisation admin passent (403 pour user standard)
- ❑ Tests isolation données passent (403 profil privé autre user)
- ❑ Tests anti-injection SQL passent